INGENIERÍA DE SISTEMAS DE IA

Construcción de un Agente IA para Mesa de Ayuda

Guía paso a paso para diseñar, estructurar y automatizar decisiones de ingeniería en soporte técnico.

El Problema a Resolver

Situación Operativa

Una organización promedio recibe decenas o cientos de incidentes de IT por día de forma desordenada:

- ⚠ "No funciona internet en el laboratorio"
- ⚠ "El sistema de cobros de la universidad está caído"
- ⚠ "No puedo acceder a mi correo institucional"

El cuello de botella

Los técnicos de soporte deben leer manualmente cada ticket, clasificarlo, asignar prioridad y elegir el responsable idóneo.

¿Podemos automatizar esta primera capa con un agente inteligente?



Objetivos del Agente de IA

Qué SÍ debe hacer

Definimos el alcance estricto de nuestro agente en fase inicial:

- ✔ **Analizar:** Extraer el contexto de incidentes en texto libre.
- ✔ **Clasificar:** Detectar automáticamente el área responsable.
- ✔ **Priorizar:** Asignar severidad en base a la criticidad.
- ✔ **Sugerir:** Indicar acciones rápidas y registrar evidencia inicial.

Qué NO hará

Las acciones operativas de alto riesgo quedan fuera del flujo inicial:

- ✖ **Reparar:** No intervendrá directamente sobre servidores físicos.
- ✖ **Configurar:** No alterará parámetros críticos de red.
- ✖ **Reiniciar:** No ejecutará scripts reactivos automáticamente.

Concepto clave: *Primero estructuramos y automatizamos decisiones simples antes de automatizar acciones de alto impacto.*

Entradas del Sistema

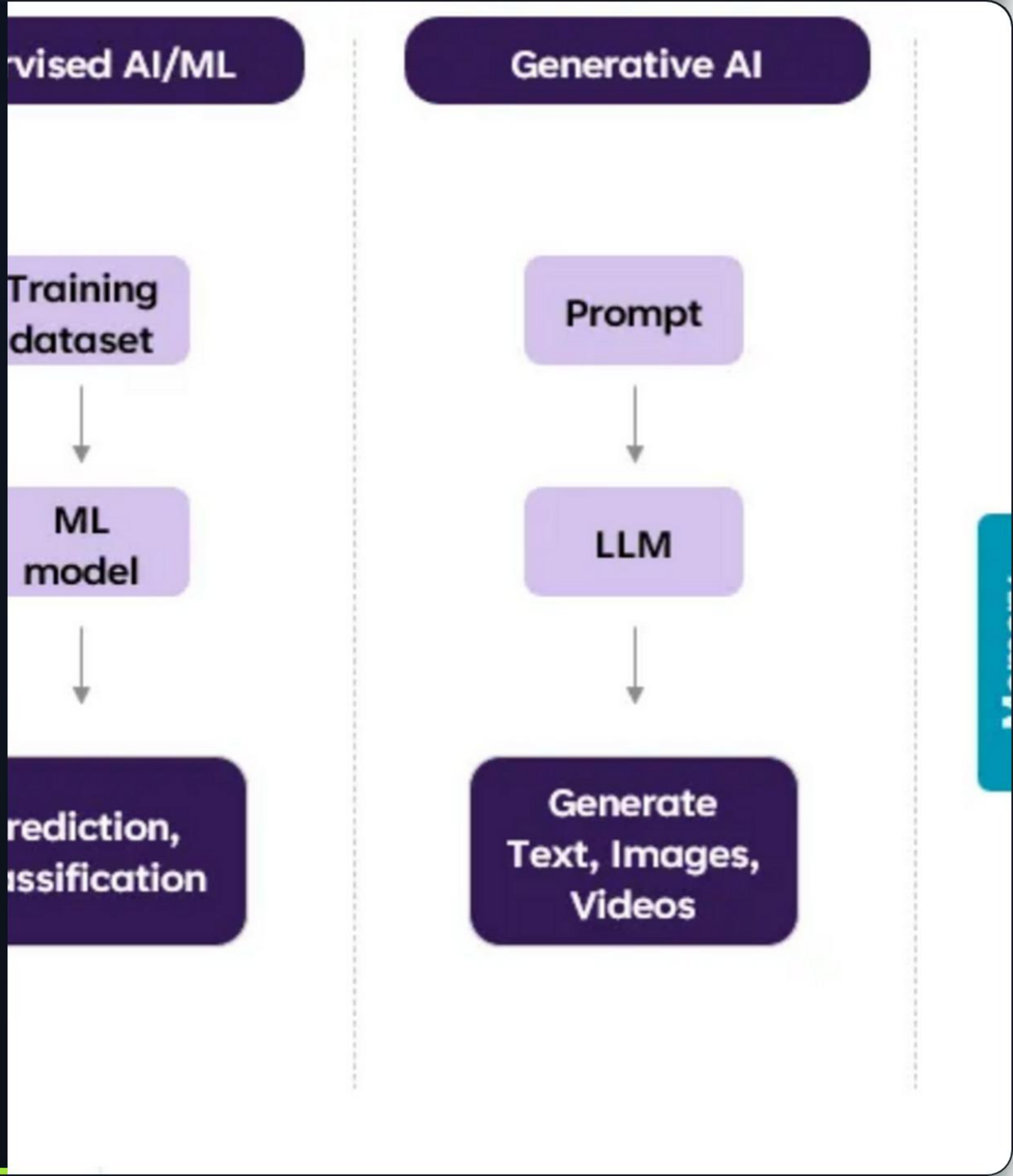
Procesamiento de Lenguaje Libre

El punto de partida es la variabilidad extrema de la comunicación humana. Las personas describen el mismo problema técnico de infinitas maneras:

```
"No puedo entrar a mi casilla de mails desde temprano"
"El sistema de cobros UNDAV no responde en caja"
"Se cayó la red inalámbrica en todo el tercer piso"
```

El Desafío de Ingeniería

No podemos programar con condicionales fijos cada variación de sintaxis. El sistema requiere semántica para interpretar la intención real del usuario.



Decisión 1: Clasificación

Redes y Conectividad

Incidencias vinculadas a Access Points, Switches, ruteo IP, cables de red o falta total de señal inalámbrica.

Sistemas Críticos

Bases de datos caídas, software institucional inaccesible (Sistemas de Gestión de Alumnos, Cobros o Académicos).

Cuentas y Accesos

Blanqueo de contraseñas de correos institucionales, restablecimiento de accesos a plataformas en la nube.

Microinformática

Computadoras lentas, fallas en impresoras físicas locales, instalación de drivers o periféricos de aula.

Objetivo final: Enrutar automáticamente el ticket al equipo de guardia correspondiente sin demora humana.

Lógica de la Clasificación

Código Base del Algoritmo

Una implementación simplificada usando substrings básicos para clasificar:

```
def clasificar_incidente(texto):
    if "internet" in texto:
        return "Redes"

    if "correo" in texto:
        return "Accesos"

    if "servidor" in texto:
        return "Sistemas"

    return "General"
```

Análisis Crítico de Ingeniería

Este enfoque tiene consecuencias directas en la confiabilidad de nuestra mesa de ayuda:

- ✔ **Ventaja:** Ejecución ultra rápida en microsegundos, consumo de CPU nulo y lógica de negocio totalmente transparente.
- ✖ **Limitación:** Depende exclusivamente de keywords exactas. Si el usuario escribe "La red está caída en el aula", fallará al clasificarlo en "Redes" por falta de la palabra "internet".

Decisión 2: Prioridad

Ponderación de Incidentes

No todos los incidentes tienen el mismo impacto operacional ni la misma urgencia. El agente debe discernir entre fallas individuales y fallas sistémicas:

PRIORIDAD ALTA

"El servidor de cobros de tesorería no responde"

PRIORIDAD MEDIA / BAJA

"Mi computadora de oficina tarda mucho en iniciar"

Asignación de Recursos

Al identificar correctamente las prioridades en base a reglas de urgencia y alcance, logramos:

- ✔ Maximizar la eficiencia de los guardias de soporte técnico.
- ✔ Reducir el SLA (Service Level Agreement) en eventos críticos que paralizan la institución.
- ✔ Evitar que tickets triviales bloqueen la atención de caídas generales de infraestructura.

Lógica de Priorización

Heurísticas de Gravedad

El motor aplica reglas heurísticas de lenguaje para etiquetar la severidad:

```
if "no responde" in texto:
    prioridad = "Alta"

if "toda el aula" in texto:
    prioridad = "Alta"

if "no puedo" in texto:
    prioridad = "Media"
```

Transformar Texto en Datos

A través de estas líneas básicas, el agente está traduciendo texto de lenguaje natural desestructurado en un indicador estructurado que determina flujos de trabajo técnicos.

Reflexión: El diseño de sistemas inteligentes parte de estructurar el caos textual en variables cuantitativas de decisión.

Recomendación de Acciones

Una vez determinada la categoría y la prioridad, el agente aporta valor inmediato inyectando instrucciones de pre-diagnóstico:

Sugerencias de Redes

- → Verificar estado de Switch local.
- → Validar energía en Access Point.
- → Correr ping de diagnóstico básico.

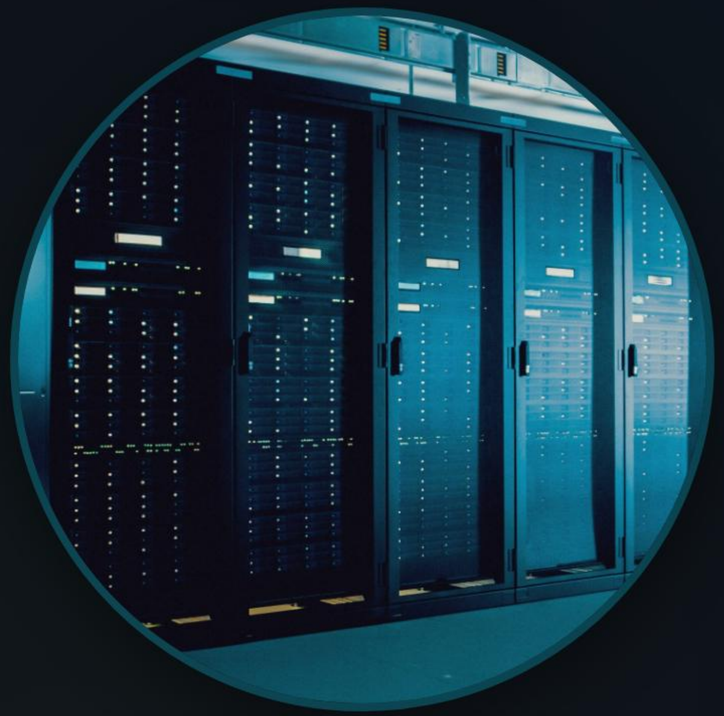
Sugerencias de Sistemas

- → Revisar pool de conexiones.
- → Verificar uso de disco y CPU.
- → Revisar logs de servidor crítico.

Sugerencias de Accesos

- → Validar estado de Active Directory.
- → Verificar bloqueos de IPs externas.
- → Revisar log de fallas OAuth.

Base de Conocimiento



¿De dónde provienen las acciones?

En el prototipo de reglas inicial, las respuestas están escritas en diccionarios estáticos de variables:

```
acciones_redes = [
    "Verificar AP",
    "Verificar switch"
]
```

Evolución de Arquitectura

En un entorno real de mesa de ayuda (como la Dirección de Sistemas de UNDAV), esta información evoluciona hacia:

- ✔ **Bases SQL / NoSQL:** Consultas dinámicas parametrizadas.
- ✔ **Wikis Técnicas / Markdown:** Parsers de documentación institucional.
- ✔ **IA Generativa (RAG):** Vectores de búsqueda en base a manuales.

Auditoría y Registro

¿Por qué persistir los datos?

Guardar cada decisión tomada por el agente de forma persistente y estructurada es vital para:

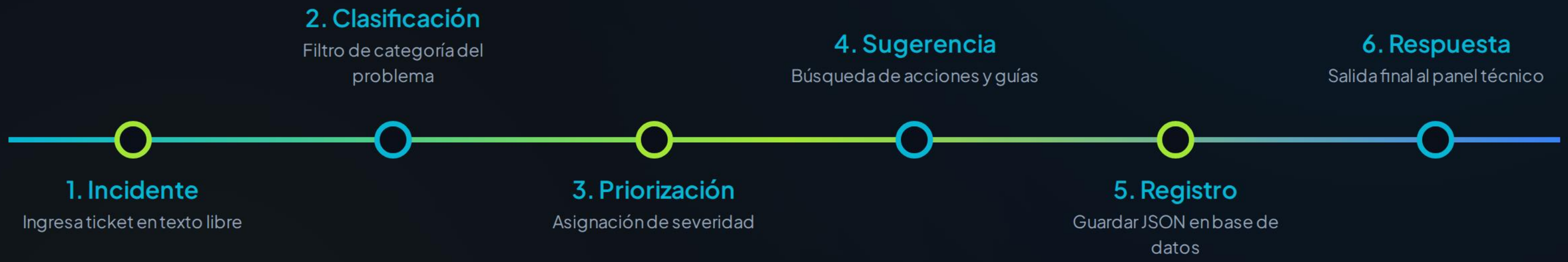
- ✔ **Trazabilidad:** Poder auditar el porqué de una clasificación errónea de prioridad.
- ✔ **Métricas de SLA:** Analizar cuántos incidentes son derivados correctamente de forma automatizada.
- ✔ **Construcción de Indicadores:** Dashboards en tiempo real de picos de incidentes por campus o área.

Payload del Registro

Ejemplo de estructura de datos JSON estandarizada que se almacena en el sistema para posterior análisis:

```
{
  "fecha": "2026-06-11",
  "categoria": "Redes",
  "prioridad": "Alta",
  "origen": "agente_ia_v1"
}
```

Flujo de Procesamiento Completo



Ejecución en Vivo

INPUT (Texto del Usuario)	CATEGORÍA	PRIORIDAD	RECOMENDACIONES
`"El servidor de cobros no responde"`	Sistemas Críticos	Alta	• Revisar servidor • Revisar logs de transacciones • Escalar al área de base de datos

Preguntas de Análisis Técnico:
¿Qué variables de decisión tomó el código? ¿Qué pasaría si la base de datos se cae pero el usuario usa la palabra "servidor de mails"? ¿Cómo evitar clasificaciones erráticas de prioridad?

Limitaciones del Enfoque de Reglas

Debilidades del Prototipo

Nuestra primera versión estructurada en reglas duras posee limitaciones severas de ingeniería:

- ✖ **Rigidez de keywords:** Falta de tolerancia al error de tipeo o sinónimos.
- ✖ **Ignorancia de contexto:** No entiende la ironía, metáforas ni dobles intenciones.
- ✖ **Nulo aprendizaje latente:** El sistema es puramente determinista; no aprende de elecciones de técnicos veteranos.

Caso de Falla Práctico

Imaginemos que ingresa este ticket real:

```
"La plataforma académica presenta intermitencias al validar usuarios"
```

El parseador actual fallará. No contiene las palabras clave como "servidor" o "internet" exactas. El agente probablemente devolverá la categoría **General** con prioridad baja, demorando una resolución importante.

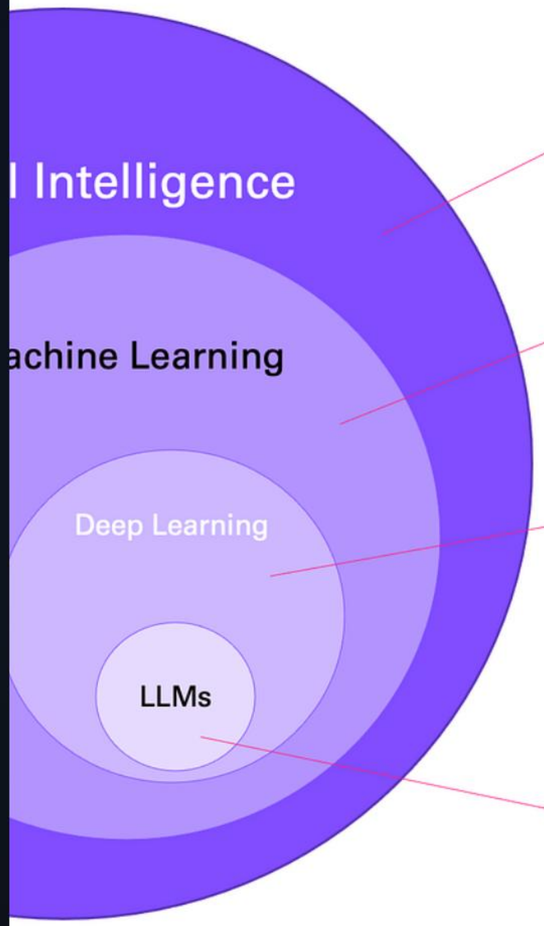
Evolución hacia Agente LLM

El Salto Generativo

Para la próxima versión, reemplazamos la lógica dura de 'if/else' por llamadas parametrizadas a un Modelo de Lenguaje de Gran Escala (LLM) como GPT, Llama o Mistral:

- ✔ **Comprensión Semántica:** Entiende metáforas, sinónimos y redacciones ambiguas de usuarios.
- ✔ **Clasificación Flexible:** Infiere el área asignando probabilidades contextuales.
- ✔ **Mejoras en Soluciones:** Capacidad de redactar guías dinámicas basadas en documentación previa estructurada.

Resultado de Ingeniería: Pasamos de un script de control básico a un agente inteligente adaptable para problemas complejos en infraestructuras universitarias reales.



Intelligent Machines
Broadly defined

Pattern Recognition
Learning general patterns fr
data

Neural Networks
Learning general patterns i
unstructured data (i.e. ima
text, audio, etc.)

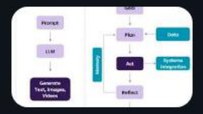
Large Language Models
Learning to understand natural
language (i.e. text)

Image Sources



https://myteltek.com/wp-content/uploads/2021/10/blog-noc.jpg

Source: myteltek.com



https://miro.medium.com/v2/resize:fit:2000/0*yY_9Qtc4YeVpVC_-.png

Source: ai.gopubby.com



https://assetphysics.com/wp-content/uploads/2025/02/AdobeStock_267083342_PFG-scaled.jpg

Source: assetphysics.com



https://miro.medium.com/v2/resize:fit:2000/1*a8LZZFns5L2VNZIUl6UasA.png

Source: medium.com